Data Types in SQL

1. Numeric data type

Data Type	Description	Example
`INT` / `INTEGER`	Whole numbers (no decimals upto 10 digit)	10
`SMALLINT`	Small-range integers(-32768 to 32767)	120
`BIGINT`	Very large integers	9223372036854775807
`TINYINT`	Very small integers (0 to 255)	100
`DECIMAL(p,s)` or `NUMERIC(p,s)`	Fixed precision numbers (p = total digits, s = digits after decimal)	123.45
`FLOAT`	Approximate decimal numbers (single precision)	12.34
`DOUBLE` / `REAL`	Approximate decimal numbers (double precision)	12345.6789
`BIT`	Boolean type (0 or 1)	1

Syantax :

```
CREATE TABLE products (
product_id INT,
price DECIMAL(10,2),
rating FLOAT
);
```

```
Start coding or generate with AI.
```

2. Character / String Data Types

Data Type	Description	Example
`CHAR(n)`	Fixed-length string (n characters)	'DELHI'
`VARCHAR(n)`	Variable-length string (up to n characters)	'Mohit Kashyap'
`TEXT`	Large text block	'This is a long paragraph...'
`NVARCHAR(n)`	Unicode variable-length string (for multilingual data)	'你好'
`NCHAR(n)`	Unicode fixed-length string	'पाठ'

Synatx :

```
CREATE TABLE students (
name VARCHAR(50),
city CHAR(10)
);
```

```
Start coding or generate with AI.
```

3. Date and Time Data Types

Data Type	Description	Example
`DATE`	Stores only date (YYYY-MM-DD)	`2025-10-16`
`TIME`	Stores only time (HH:MM:SS)	`13:45:30`
`DATETIME`	Stores both date and time	`2025-10-16 13:45:30`
`TIMESTAMP`	Stores date & time with automatic updates	`2025-10-16 13:45:30`
`YEAR`	Stores year (2 or 4 digits)	`2025`

Syntax :

```
CREATE TABLE orders (
order_id INT,
order_date DATE,
delivery_time TIME,
created_at DATETIME
);
```

```
Start coding or generate with AI.
```

4. Binary Data Types

Data Type	Description	Example
`BINARY(n)`	Fixed-length binary data	101010
`VARBINARY(n)`	Variable-length binary data	0101
`BLOB`	Large binary object (Binary Large Object)	image file, PDF

Syntax :

```
CREATE TABLE files (
file_id INT,
file_data BLOB
);
```

```
Start coding or generate with AI.
```

5. Miscellaneous / Advanced Types

Data Type	Description	Example
`BOOLEAN`	True or False value	TRUE
`ENUM('val1','val2',...)`	Predefined set of values	'Active', 'Inactive'
`SET('a','b','c',...)`	Multiple allowed values from a list	'a,b'
`JSON`	Stores JSON formatted data	`{"name":"Mohit","age":21}`
`XML`	Stores XML data	`<student><name>Mohit</name></student>`

Syntax :

```
CREATE TABLE users (
id INT,
status ENUM('Active', 'Inactive'),
preferences JSON
);
```

```
Start coding or generate with AI.
```

🧩 SQL Operators

- SQL Operators are special symbols or keywords used to perform operations on data — such as comparison, arithmetic, logical tests, and pattern matching

Types of SQL Operators

Category	Description
1. Arithmetic Operators	Perform mathematical operations
2. Comparison Operators	Compare two values

Category	Description
3. Logical Operators	Combine multiple conditions
4. Bitwise Operators	Perform bit-level operations (rare)
5. Special Operators	Used for patterns, ranges, or null checks

1 Arithmetic Operators

Operator	Description	Example	Result
`+`	Addition	`SELECT 10 + 5;`	15
`-`	Subtraction	`SELECT 10 - 5;`	5
`*`	Multiplication	`SELECT 10 * 5;`	50
`/`	Division	`SELECT 10 / 5;`	2
`%`	Modulus (remainder)	`SELECT 10 % 3;`	1

```
56
57 •  select*from employees;
58
```

Result Grid | Filter Rows: | Edit: | Expo

EmpID	EmpName	Age	DepartmentID	Salary	City
1	Amit	25	101	35000.00	Delhi
2	Priya	28	102	42000.00	Mumbai
3	Ravi	30	103	50000.00	Bangalore
4	Neha	22	101	30000.00	Delhi
5	Rahul	35	104	60000.00	Pune
6	Sneha	29	103	55000.00	Chennai
7	Arjun	31	102	48000.00	Mumbai
NULL	NULL	NULL	NULL	NULL	NULL

Employee table

```
60
61      --   ARITHMATICS OPERATION -----
62
63      -- - Addition
64 •  SELECT empname, salary + 2000 AS new_salary
65      FROM employees;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Conte

empname	new_salary
Amit	37000.00
Priya	44000.00
Ravi	52000.00
Neha	32000.00
Rahul	62000.00
Sneha	57000.00
Arjun	50000.00

Perform addtion operation on salary

```
56
57 •  select*from employees;
58
```

Result Grid | Filter Rows: | Edit: | Expo

EmpID	EmpName	Age	DepartmentID	Salary	City
1	Amit	25	101	35000.00	Delhi
2	Priya	28	102	42000.00	Mumbai
3	Ravi	30	103	50000.00	Bangalore
4	Neha	22	101	30000.00	Delhi
5	Rahul	35	104	60000.00	Pune
6	Sneha	29	103	55000.00	Chennai
7	Arjun	31	102	48000.00	Mumbai
NULL	NULL	NULL	NULL	NULL	NULL

Employee table

```
68     -- subtraction
69 •  SELECT empname, salary - 2000 AS new_salary
70     FROM employees;
71
```

Result Grid | Filter Rows: | Export: | Wrap Cell Cont

empname	new_salary
Amit	33000.00
Priya	40000.00
Ravi	48000.00
Neha	28000.00
Rahul	58000.00
Sneha	53000.00
Arjun	46000.00

Perform subtraction operation on salary with name as New_salary

```
56
57 •  select*from employees;
58
```

Result Grid | Filter Rows: | Edit: | Expo

EmpID	EmpName	Age	DepartmentID	Salary	City
1	Amit	25	101	35000.00	Delhi
2	Priya	28	102	42000.00	Mumbai
3	Ravi	30	103	50000.00	Bangalore
4	Neha	22	101	30000.00	Delhi
5	Rahul	35	104	60000.00	Pune
6	Sneha	29	103	55000.00	Chennai
7	Arjun	31	102	48000.00	Mumbai
NULL	NULL	NULL	NULL	NULL	NULL

Employee table

```
75    -- multiplication
76 •  SELECT empname, salary * 0.13 AS new_salary
77    FROM employees;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Co

empname	new_salary
Amit	33000.00
Priya	40000.00
Ravi	48000.00
Neha	28000.00
Rahul	58000.00
Sneha	53000.00
Arjun	46000.00

Perform multiplication operation on salary

```
56
57 •  select*from employees;
58
```

Result Grid | Filter Rows: | Edit: | Exp

EmpID	EmpName	Age	DepartmentID	Salary	City
1	Amit	25	101	35000.00	Delhi
2	Priya	28	102	42000.00	Mumbai
3	Ravi	30	103	50000.00	Bangalore
4	Neha	22	101	30000.00	Delhi
5	Rahul	35	104	60000.00	Pune
6	Sneha	29	103	55000.00	Chennai
7	Arjun	31	102	48000.00	Mumbai
NULL	NULL	NULL	NULL	NULL	NULL

Employee table

```
80
81    -- Division
82 •  SELECT empname, salary / 10 AS new_salary
83    FROM employees;
```

Result Grid | Filter Rows: | Export: | Wra

empname	new_salary
Amit	3500.000000
Priya	4200.000000
Ravi	5000.000000
Neha	3000.000000
Rahul	6000.000000
Sneha	5500.000000
Arjun	4800.000000

Perform division operation on salary

```
56
57 •  select*from employees;
58
```

Result Grid | Filter Rows: | Edit: | Exp

EmpID	EmpName	Age	DepartmentID	Salary	City
1	Amit	25	101	35000.00	Delhi
2	Priya	28	102	42000.00	Mumbai
3	Ravi	30	103	50000.00	Bangalore
4	Neha	22	101	30000.00	Delhi
5	Rahul	35	104	60000.00	Pune
6	Sneha	29	103	55000.00	Chennai
7	Arjun	31	102	48000.00	Mumbai
NULL	NULL	NULL	NULL	NULL	NULL

Employee table

```
86  -- Modules % (reminder)
87 •  SELECT empname, salary % 10 AS new_salary
88  FROM employees;
89
```

Result Grid | Filter Rows: | Export: | Wrap Cell

empname	new_salary
Amit	0.00
Priya	0.00
Ravi	0.00
Neha	0.00
Rahul	0.00
Sneha	0.00
Arjun	0.00

Perform modules operation on salary

```
Start coding or generate with AI.
```

2 Comparison Operators

Operator	Description	Example
`=`	Equal to	`WHERE city = 'Delhi'`
`!=` or `<>`	Not equal to	`WHERE city <> 'Mumbai'`
`>`	Greater than	`WHERE age > 25`

Operator	Description	Example
`<`	Less than	`WHERE salary < 50000`
`>=`	Greater than or equal to	`WHERE marks >= 40`
`<=`	Less than or equal to	`WHERE date <= '2025-10-16'`

```
56
57 •  select*from employees;
58
```

Result Grid | Filter Rows: | Edit: | Exp

EmpID	EmpName	Age	DepartmentID	Salary	City
1	Amit	25	101	35000.00	Delhi
2	Priya	28	102	42000.00	Mumbai
3	Ravi	30	103	50000.00	Bangalore
4	Neha	22	101	30000.00	Delhi
5	Rahul	35	104	60000.00	Pune
6	Sneha	29	103	55000.00	Chennai
7	Arjun	31	102	48000.00	Mumbai
NULL	NULL	NULL	NULL	NULL	NULL

Employee table

```
91      -- ---- COMPARISION OPERATOR --------
92
93      -- Equal operator =
94 •    SELECT empname, city
95      FROM employees WHERE city = 'Delhi';
```

Result Grid | Filter Rows: | Export: | W

empname	city
Amit	Delhi
Neha	Delhi

Perform equal operator on city

```
56
57 •  select*from employees;
58
```

Result Grid | Filter Rows: | Edit: | Exp

EmpID	EmpName	Age	DepartmentID	Salary	City
1	Amit	25	101	35000.00	Delhi
2	Priya	28	102	42000.00	Mumbai
3	Ravi	30	103	50000.00	Bangalore
4	Neha	22	101	30000.00	Delhi
5	Rahul	35	104	60000.00	Pune
6	Sneha	29	103	55000.00	Chennai
7	Arjun	31	102	48000.00	Mumbai
NULL	NULL	NULL	NULL	NULL	NULL

Employee table

```
97
98    -- Does not Equal operator != ,<>
99 •  SELECT empname, city
100   FROM employees WHERE city != 'Delhi';
101
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

empname	city
Priya	Mumbai
Ravi	Bangalore
Rahul	Pune
Sneha	Chennai
Arjun	Mumbai

Perform Not equal operator on city

```
56
57 •    select*from employees;
58
```

Result Grid | Filter Rows: | Edit: | Exp

EmpID	EmpName	Age	DepartmentID	Salary	City
1	Amit	25	101	35000.00	Delhi
2	Priya	28	102	42000.00	Mumbai
3	Ravi	30	103	50000.00	Bangalore
4	Neha	22	101	30000.00	Delhi
5	Rahul	35	104	60000.00	Pune
6	Sneha	29	103	55000.00	Chennai
7	Arjun	31	102	48000.00	Mumbai
NULL	NULL	NULL	NULL	NULL	NULL

Employee table

```
103     -- Greater Than operator <
104 •   SELECT empname, age
105     FROM employees WHERE age > 25;
106
```

Result Grid | Filter Rows: | Export:

empname	age
Priya	28
Ravi	30
Rahul	35
Sneha	29
Arjun	31

Perform Gerater operation on age

```
56
57 •  select*from employees;
58
```

Result Grid | Filter Rows: | Edit: | Exp

EmpID	EmpName	Age	DepartmentID	Salary	City
1	Amit	25	101	35000.00	Delhi
2	Priya	28	102	42000.00	Mumbai
3	Ravi	30	103	50000.00	Bangalore
4	Neha	22	101	30000.00	Delhi
5	Rahul	35	104	60000.00	Pune
6	Sneha	29	103	55000.00	Chennai
7	Arjun	31	102	48000.00	Mumbai
NULL	NULL	NULL	NULL	NULL	NULL

Employee table

```
108     -- Less Than operator <
109 •   SELECT empname, salary
110     FROM employees WHERE salary < 50000;
111
112
```

Result Grid | Filter Rows: | Export: | Wrap Cell Con

empname	salary
Amit	35000.00
Priya	42000.00
Neha	30000.00
Arjun	48000.00

Perform Less than operator on salary

```
56
57 •  select*from employees;
58
```

Result Grid | Filter Rows: | Edit: | Exp

EmpID	EmpName	Age	DepartmentID	Salary	City
1	Amit	25	101	35000.00	Delhi
2	Priya	28	102	42000.00	Mumbai
3	Ravi	30	103	50000.00	Bangalore
4	Neha	22	101	30000.00	Delhi
5	Rahul	35	104	60000.00	Pune
6	Sneha	29	103	55000.00	Chennai
7	Arjun	31	102	48000.00	Mumbai
NULL	NULL	NULL	NULL	NULL	NULL

Employee table

```
118     -- Less than equal to  >=
119 •   SELECT empname, age
120     FROM employees WHERE age <= 25;
121
122
```

Result Grid | Filter Rows: | Export: | Wra

empname	age
Amit	25
Neha	22

Perform Less than equal to on age

```
Start coding or generate with AI.
```

3 Logical Operators

Operator	Description	Example
`AND`	Returns TRUE if both conditions are TRUE	`WHERE age > 18 AND city = 'Delhi'`
`OR`	Returns TRUE if any condition is TRUE	`WHERE city = 'Delhi' OR city = 'Mumbai'`
`NOT`	Reverses condition result	`WHERE NOT city = 'Delhi'`

```
56
57 •  select*from employees;
58
```

Result Grid | Filter Rows: | Edit: | Exp

EmpID	EmpName	Age	DepartmentID	Salary	City
1	Amit	25	101	35000.00	Delhi
2	Priya	28	102	42000.00	Mumbai
3	Ravi	30	103	50000.00	Bangalore
4	Neha	22	101	30000.00	Delhi
5	Rahul	35	104	60000.00	Pune
6	Sneha	29	103	55000.00	Chennai
7	Arjun	31	102	48000.00	Mumbai
NULL	NULL	NULL	NULL	NULL	NULL

Employee table

```
124  -- ------ LOGICAL OPERATOR ---------
125
126  -- AND
127 •  select age,city from employees WHERE age > 18 AND city = 'Delhi';
128
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

age	city
25	Delhi
22	Delhi

Perform AND operation on city

```
56
57 •  select*from employees;
58
```

Result Grid | Filter Rows: | Edit: | Exp

EmpID	EmpName	Age	DepartmentID	Salary	City
1	Amit	25	101	35000.00	Delhi
2	Priya	28	102	42000.00	Mumbai
3	Ravi	30	103	50000.00	Bangalore
4	Neha	22	101	30000.00	Delhi
5	Rahul	35	104	60000.00	Pune
6	Sneha	29	103	55000.00	Chennai
7	Arjun	31	102	48000.00	Mumbai
NULL	NULL	NULL	NULL	NULL	NULL

Employee table

```
131
132     -- OR
133 •   SELECT Age, City FROM Employees WHERE City = 'Delhi' OR City = 'Mumbai';
134
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Age	City
25	Delhi
28	Mumbai
22	Delhi
31	Mumbai

Perform OR operation on city

```
56
57 •  select*from employees;
58
```

Result Grid | Filter Rows: | Edit: | Exp

EmpID	EmpName	Age	DepartmentID	Salary	City
1	Amit	25	101	35000.00	Delhi
2	Priya	28	102	42000.00	Mumbai
3	Ravi	30	103	50000.00	Bangalore
4	Neha	22	101	30000.00	Delhi
5	Rahul	35	104	60000.00	Pune
6	Sneha	29	103	55000.00	Chennai
7	Arjun	31	102	48000.00	Mumbai
NULL	NULL	NULL	NULL	NULL	NULL

Employee table

```
135  -- NOT
136 •  SELECT Age, City FROM Employees WHERE NOT City = 'Delhi';
137
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Age	City
28	Mumbai
30	Bangalore
35	Pune
29	Chennai
31	Mumbai

Perform NOT operation on city

4 Bitwise Operators (rarely used)

Operator	Description	Example
&	Bitwise AND	`SELECT 5 & 3;` → 1
|	Bitwise OR	`SELECT 5 | 3;` → 7
^	Bitwise XOR	`SELECT 5 ^ 3;` → 6

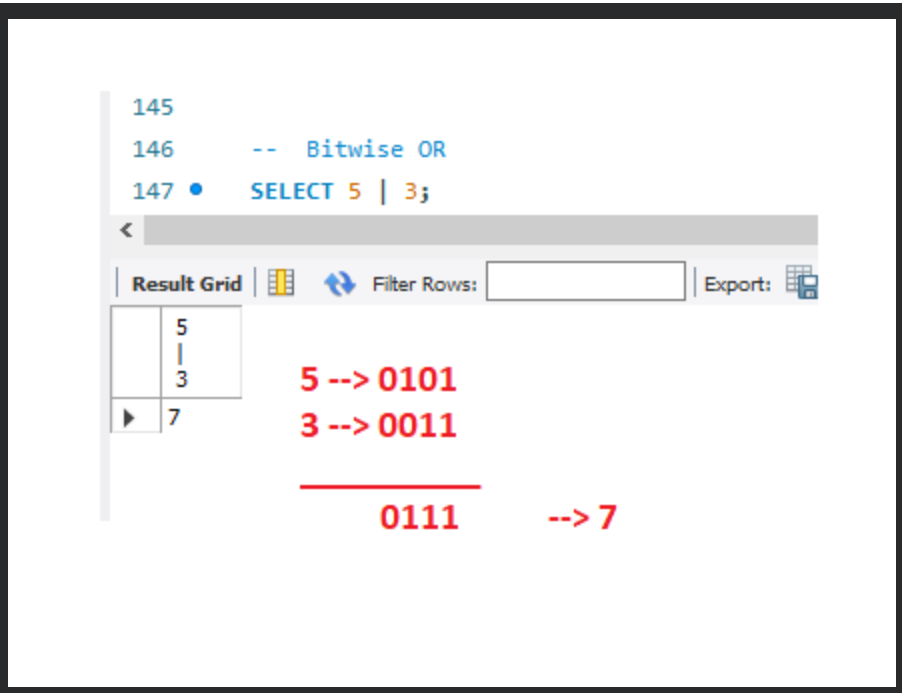
```
140  -- ------- BITWISE OPERATOR (rarely used)  ------
141
142  -- Bitwise AND
143  SELECT 5 & 3;
```

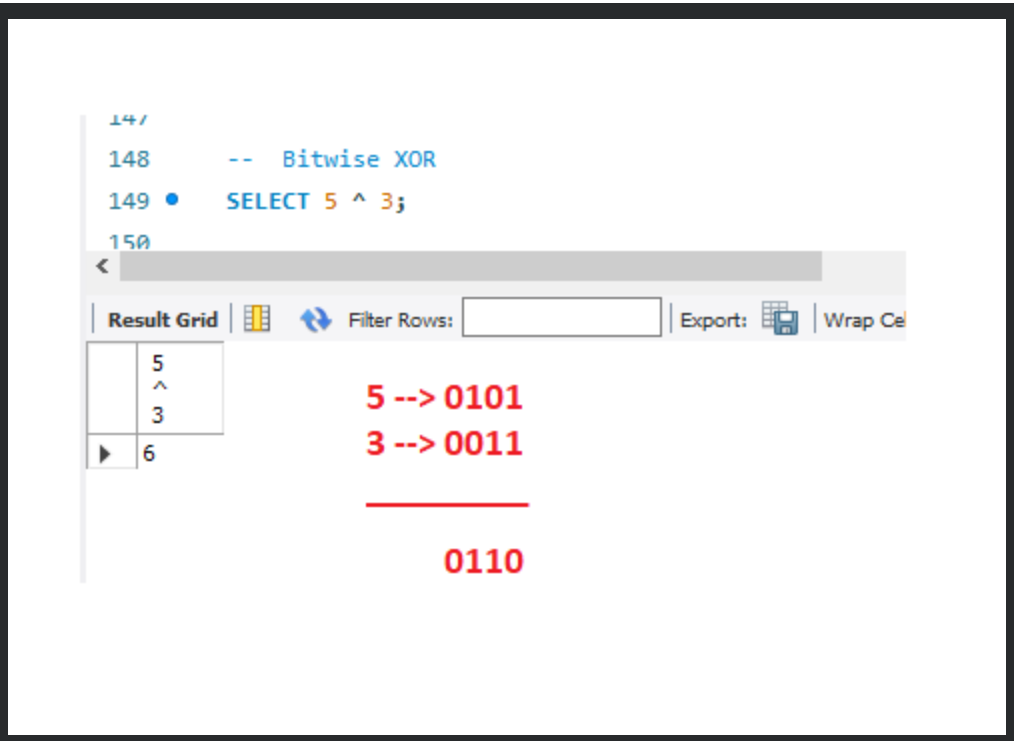
Result Grid | Filter Rows: | Export: | Wrap Cell Content:

5 & 3
1

5 --> 0101
3 --> 0011

0001


145
146 -- Bitwise OR
147 SELECT 5 | 3;
Result Grid
Filter Rows:
Export:
5 | 3
7
5 --> 0101
3 --> 0011
0111 --> 7



5 Special Operators

Operator	Description	Example
`BETWEEN`	Checks if value lies within a range	`WHERE salary BETWEEN 30000 AND 50000`
`IN`	Matches a list of values	`WHERE city IN ('Delhi','Mumbai','Pune')`
`IS NULL`	Checks for NULL values	`WHERE email IS NULL`
`IS NOT NULL`	Checks for non-NULL values	`WHERE phone IS NOT NULL`
`EXISTS`	Checks if subquery returns any row	`WHERE EXISTS (SELECT * FROM orders)`
`ANY`	Compares with any value in a list	`WHERE salary > ANY (SELECT salary FROM interns)`
`ALL`	Compares with all values in a list	`WHERE salary > ALL (SELECT salary FROM interns)`

Operator	Description	Example
`LIKE`	Matches a pattern using wildcards	`WHERE name LIKE 'M%'`

```
151
152     --  ---- SPECIAL OPERATOR -----
153
154     -- Between Operator
155     SELECT empname, salary FROM Employees WHERE salary BETWEEN 30000 AND 50000;
156
157
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

empname	salary
Amit	35000.00
Priya	42000.00
Ravi	50000.00
Neha	30000.00
Arjun	48000.00

BETWEEN includes the boundary values as well as.

Means -->(3000 and 5000)

Boundary values

```
165
166     -- IN Operator
167
168 •   SELECT empname, salary FROM Employees WHERE city IN ('Delhi','Mumbai','Pune');
169
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

empname	salary
Amit	35000.00
Priya	42000.00
Neha	30000.00
Rahul	60000.00
Arjun	48000.00

The IN operator allows you to specify multiple values in a WHERE clause.

It's like saying: "match any of these values."

Double-click (or enter) to edit

Double-click (or enter) to edit

```
Start coding or generate with AI.
```

6 Wildcard Operators (used with LIKE)

LIKE

- It is filter the records from the column based on the pattern specified in the sql query.
- Like is used in the where clause with the following statement
  - SELECT
  - UPDATE
  - DELETE

There are four conjunction that is used with Like Operator.

Wildcard	Meaning	Example
`%`	Zero or more characters	`'A%'` → Starts with A
`_`	Exactly one character	`'S_m'` → "Sam", "Sim"
`[ ]`	Any single character inside brackets	`'[AB]%'` → Starts with A or B
`[^ ]`	Excludes characters inside brackets	`'[^A]%'` → Doesn't start with A

1. Percentage %

- It represent zero ,one ,multiple ,character. But % length is not fixed.

Pattern	Meaning	Example Match
`LIKE '%a%'`	**'a' is anywhere** in the string	`data`, `apple`, `banana`
`LIKE '%a'`	**'a' is at the end**	`india`, `panda`
`LIKE 'a%'`	**'a' is at the start**	`apple`, `animal`

Pattern	Meaning	Example Match
`LIKE 'a%b'`	Starts with 'a' and ends with 'b'	`acb`, `a123b`, `ab`

```
224 •   SELECT * FROM WORDS;
225
```

Result Grid | Filter Rows: | Edit:

id	word
1	Apple
2	Banana
3	Ball
4	Cat
5	Candle
6	Cow
7	Dog
8	yellow
9	xenon
10	zebra
11	air
12	bat
13	axe
NULL	NULL

Initial Words table

```
219   --  ------------------------ LIKE OPERATOR ----------------
220
221   -- Percentage ------
222
223 •   SELECT * FROM Words WHERE word LIKE '%an%';
224
```

Result Grid | Filter Rows: | Edit: | Export/Import:

id	word
2	Banana
5	Candle
NULL	NULL

%an% ---> anywhere in the string

```
224    SELECT * FROM WORDS;
225
```

Result Grid | Filter Rows: | Edit:

id	word
1	Apple
2	Banana
3	Ball
4	Cat
5	Candle
6	Cow
7	Dog
8	yellow
9	xenon
10	zebra
11	air
12	bat
13	axe
NULL	NULL

Initial Words table

```
225
226    SELECT * FROM Words WHERE word LIKE 'D%';
227
```

Result Grid | Filter Rows: | Edit: | Export/

id	word
7	Dog
NULL	NULL

D% --> select string which start with D

```
224 •   SELECT * FROM WORDS;
225
```

Result Grid | Filter Rows: | Edit:

id	word
1	Apple
2	Banana
3	Ball
4	Cat
5	Candle
6	Cow
7	Dog
8	yellow
9	xenon
10	zebra
11	air
12	bat
13	axe
NULL	NULL

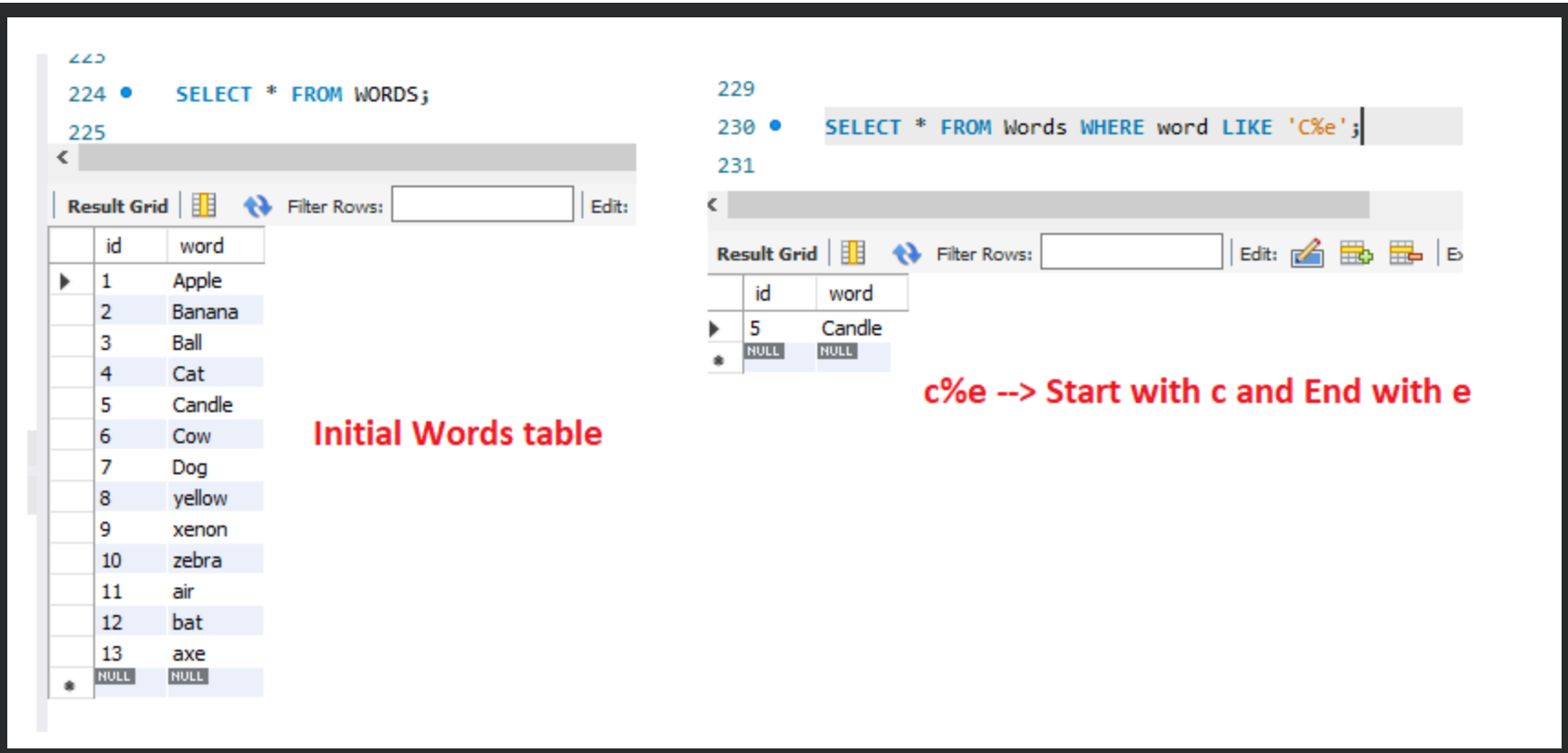
Initial Words table

```
228 •   SELECT * FROM Words WHERE word LIKE '%t';
```

Result Grid | Filter Rows: | Edit:

id	word
4	Cat
12	bat
NULL	NULL

%t --> Select String which end with t



2. Underscore _

- It represent zero ,one ,multiple ,character, have a fixed length.

Pattern	Meaning	Example Matches
`LIKE '____'`	String has **exactly 4 characters**	`data`, `mohit`, `abcd`
`LIKE '_a%'`	One character **before 'a'**, anything after	`raju`, `mahi`, `ta`
`LIKE '%a_'`	Any number of characters **before 'a'**, and **exactly one after**	`cat`, `data`, `panda`

Pattern	Meaning	Example Matches
`LIKE '_%ab'`	One character at start, **anything between**, ends with **'ab'**	`kab`, `tab`, `xcab`

```
224 •  SELECT * FROM WORDS;
225
```

Result Grid | Filter Rows: | Edit:

id	word
1	Apple
2	Banana
3	Ball
4	Cat
5	Candle
6	Cow
7	Dog
8	yellow
9	xenon
10	zebra
11	air
12	bat
13	axe
NULL	NULL

Initial Words table

```
235  -- ------- Underscore ---------
236
237 •  SELECT * FROM Words WHERE word LIKE '_a%';
```

Result Grid | Filter Rows: | Edit:

id	word
2	Banana
3	Ball
4	Cat
5	Candle
12	bat
NULL	NULL

_a% --> Exact one character before the a